Sesión 5

Modelos Clásicos para NLP: Naive Bayes, SVM, HMM, MaxEnt y CRF

Carlos César Sánchez Coronel

NLP

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

¿Por qué estudiar modelos clásicos?

- Aunque el deep learning domina la investigación, los modelos clásicos siguen siendo relevantes por:
  - **Eficiencia con datos pequeños**: funcionan bien con miles de ejemplos.
  - **Interpretabilidad**: sus decisiones son más fáciles de explicar y depurar.
  - **Línea base obligatoria**: todo proyecto serio debe comparar contra ellos.
  - **Producción en entornos restringidos**: bajo costo computacional.

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Fundamento matemático

Teorema de Bayes

$$P(c|d) = \frac{P(c)\prod_{i=1}^{n} P(w_i|c)}{P(d)}$$

con supuesto de independencia condicional:

$$P(\mathbf{w}|c) = \prod_i P(w_i|c)$$

- Variantes: **MultinomialNB** (frecuencias), **BernoulliNB** (presencia/ausencia), **GaussianNB** (continuas).

Estimación de parámetros

Suavizado Laplace

$$P(w|c) = \frac{\text{freq}(w, c) + \alpha}{\sum_{w' \in V} \text{freq}(w', c) + \alpha |V|}$$

- $\alpha = 1$ es común (add-1).
- Evita probabilidades cero para palabras no vistas.

Ejemplo numérico

Corpus:

- d1: .^el gato caza ratones" (animales)
- d2: .^el perro persigue gatos" (animales)
- d3: .^el coche acelera rápido" (vehículos)

Clasificar "gato rápidoüsando MultinomialNB con $\alpha = 1$.

Animales

$$P(\text{animal}) = \frac{2}{3}$$

$$P(\text{gato}|\text{animal}) = \frac{1+1}{6+10} = 0{,}125$$

$$P(\text{rápido}|\text{animal}) = \frac{0+1}{6+10} = 0{,}0625$$

$$\text{score} \propto 0{,}00521$$

Vehículos

$$P(\text{vehículo}) = \frac{1}{3}$$

$$P(\text{gato}|\text{vehículo}) = \frac{0+1}{4+10} = 0{,}0714$$

$$P(\text{rápido}|\text{vehículo}) = \frac{1+1}{4+10} = 0{,}1429$$

$$\text{score} \propto 0{,}00340$$

Clasificación: **animales**.

Ventajas y limitaciones

Ventajas

- Extremadamente rápido.
- Funciona bien con datos pequeños.
- Maneja alta dimensionalidad.
- Interpretable.

Limitaciones

- Asume independencia (violada en lenguaje).
- No captura relaciones entre palabras.

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

SVM lineal para texto

Formulación

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i$$

sujeto a $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$.

- C controla penalización por errores.
- Para texto, kernel lineal suele ser suficiente por alta dimensionalidad.

¿Por qué SVM funciona bien en texto?

- **Alta dimensionalidad**: los textos tienen miles de características, SVM maneja $p \gg n$.
- **Representaciones dispersas**: BOW/TF-IDF son esparsas, SVM las maneja eficientemente.
- **Margen máximo**: buena generalización.
- **Kernel lineal suficiente**: los datos de texto suelen ser aproximadamente separables linealmente.

Pipeline típico con scikit-learn

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.pipeline import make_pipeline

docs = ["el gato caza ratones", "el perro persigue gatos", "el coche acelera r pido"]
y = ["animales", "animales", "veh culos"]

pipeline = make_pipeline(
    TfidfVectorizer(),
    LinearSVC(C=1.0, class_weight='balanced')
)
pipeline.fit(docs, y)
```

Ventajas y limitaciones de SVM

Ventajas

- Robusto, buen rendimiento.
- Maneja desbalance con pesos.
- Bueno para datos de alta dimensión.

Limitaciones

- Menos interpretable que Naive Bayes.
- Requiere normalización de características.

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Definición formal de HMM

Componentes

- Estados ocultos $Q = \{q_1, \ldots, q_N\}$ (ej. etiquetas POS).
- Observaciones $O = \{o_1, \ldots, o_T\}$ (ej. palabras).
- Matriz de transición $A = [a_{ij}]$ con $a_{ij} = P(q_t = j | q_{t-1} = i)$.
- Matriz de emisión $B = [b_j(k)]$ con $b_j(k) = P(o_t = k | q_t = j)$.
- Distribución inicial $\pi = [\pi_i]$ con $\pi_i = P(q_1 = i)$.

La propiedad de Markov

Propiedad de Markov de primer orden

$$P(q_{t+1}|q_t, q_{t-1}, \ldots, q_1) = P(q_{t+1}|q_t)$$

El estado futuro solo depende del estado actual.

- Limitación: no considera contexto más amplio.

Tres problemas fundamentales en HMM

Problema	**Algoritmo**
Evaluación: $P(O\|\lambda)$	Forward
Decodificación: mejor secuencia de estados	Viterbi
Aprendizaje: estimar $\lambda = (A, B, \pi)$	Baum-Welch (EM)

POS tagging con HMM en NLTK

```
import nltk
from nltk.tag import HiddenMarkovModelTrainer
from nltk.corpus import treebank

train_data = treebank.tagged_sents()[:3900]
tagger = HiddenMarkovModelTrainer().train_supervised(train_data)

sentence = "The cat sits on the mat".split()
tags = tagger.tag(sentence)
print(tags)  # [('The', 'DT'), ('cat', 'NN'), ...]
```

Aplicaciones y limitaciones de HMM

- **Aplicaciones**: POS tagging, NER, reconocimiento de voz, segmentación.
- **Ventajas**: modelo generativo, interpretable, funciona con datos etiquetados.
- **Limitaciones**: asume independencia de observaciones dado el estado, dependencias de primer orden.

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Fundamento matemático

Probabilidad condicional

$$P(c|d) = \frac{\exp\left(\sum_j \lambda_j f_j(c,d)\right)}{\sum_{c'} \exp\left(\sum_j \lambda_j f_j(c',d)\right)}$$

donde f_j son funciones características arbitrarias.

- Equivalente a regresión logística multinomial.
- No asume independencia; puede incluir características correlacionadas.

Características (features)

- Pueden ser arbitrarias: unigramas, bigramas, contexto, sufijos, etc.
- Ejemplo: $f(\text{positivo}, d) = 1$ si la palabra .^{ex}celente.^{a}parece en d.
- Permite modelar dependencias complejas sin supuestos de independencia.

Entrenamiento y regularización

Log-verosimilitud negativa con regularización

$$\mathcal{L} = -\sum_i \log P(c_i|d_i) + \frac{1}{2C}\sum_j \lambda_j^2$$

- Optimización con gradiente descendente o L-BFGS.
- Regularización L2 (Ridge) o L1 (Lasso) evitan overfitting.

Ejemplo con scikit-learn

```
from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import TfidfVectorizer

X = ["buena pel cula", "mala pel cula", "excelente actuaci n"]
y = ["positivo", "negativo", "positivo"]

vectorizer = TfidfVectorizer()
X_vec = vectorizer.fit_transform(X)
clf = LogisticRegression(C=1.0, penalty='l2')
clf.fit(X_vec, y)
```

Ventajas y limitaciones de MaxEnt

- **Ventajas**: modelo discriminativo, flexible, no asume independencia.
- **Limitaciones**: no modela secuencias explícitamente (ignora dependencias entre etiquetas consecutivas).

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

De HMM a CRF

- HMM: generativo, modela $P(X, Y)$, asume independencia de observaciones.
- CRF: discriminativo, modela $P(Y|X)$ directamente, permite características arbitrarias y dependencias contextuales.

Definición formal de CRF

$$P(\mathbf{Y}|\mathbf{X}) = \frac{1}{Z(\mathbf{X})} \exp\left(\sum_{t=1}^{T} \sum_{k} \lambda_k f_k(y_t, y_{t-1}, \mathbf{X}, t)\right)$$

- f_k: funciones características (pueden depender de toda la secuencia de entrada).
- λ_k: pesos aprendidos.
- $Z(\mathbf{X})$: función de partición (normalización).

Tipos de características

- **De estado**: relación etiqueta actual con observaciones (ej. "palabra termina en -ción").
- **De transición**: relación etiqueta actual con anterior (ej. .^{es} poco probable que DT siga a JJ").

Implementación con sklearn-crfsuite

```
import sklearn_crfsuite
from sklearn_crfsuite import metrics

def word2features(sent, i):
    word = sent[i][0]
    return {
        'word': word,
        'is_capitalized': word[0].isupper(),
        'suffix-2': word[-2:],
        'prev_word': '' if i==0 else sent[i-1][0],
    }

X_train = [word2features(s, i) for s in train_sents ...]
crf = sklearn_crfsuite.CRF(algorithm='lbfgs', c1=0.1, c2=0.1)
crf.fit(X_train, y_train)
```

Ventajas de CRF sobre HMM

- No asume independencia de las observaciones.
- Puede incluir características arbitrarias y superpuestas.
- Mejor rendimiento en tareas como NER y chunking.
- Útil cuando se dispone de suficientes datos y características ricas.

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Tabla comparativa

Modelo	**Tipo**	**Velocidad**	**Interpretabilidad**
Naive Bayes	Generativo	Muy rápida	Alta
SVM (lineal)	Discriminativo	Rápida	Media
HMM	Generativo secuencial	Media	Alta
MaxEnt	Discriminativo	Media	Media
CRF	Discriminativo secuencial	Lenta	Baja

¿Cuándo usar cada uno?

- **Clasificación simple**: Naive Bayes (baseline), SVM (mejor rendimiento).
- **Etiquetado secuencial**: HMM (pocos datos), CRF (características ricas).
- **Cuando la independencia es insostenible**: MaxEnt o CRF.

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Problema

- Corpus de reseñas de productos con calificación (1-5 estrellas).
- Tareas:
  1. Clasificación de sentimiento (positivo/negativo).
  2. Extracción de aspectos mencionados (NER: batería, pantalla, precio).

Solución con modelos clásicos

Clasificación

- SVM con TF-IDF.
- MaxEnt con características adicionales.
- Comparar con Naive Bayes.

NER

- HMM simple.
- CRF con características ricas (contexto, sufijos).

Resultados esperados: SVM supera a NB; CRF supera a HMM.

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Preguntas típicas

- ¿Cuándo usar Naive Bayes vs SVM? (datos pequeños vs. robustez)
- ¿Por qué SVM con kernel lineal funciona bien en texto?
- Explica el algoritmo de Viterbi.
- Diferencia entre HMM y CRF.
- ¿Qué es el suavizado Laplace?
- ¿Cómo manejarías desbalance en SVM?

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Próximos pasos

- **Sesión 4** (Word Embeddings): se pueden usar como características densas en estos modelos.
- **Sesión 6** (RNN/LSTM): superan a HMM/CRF en secuencias, pero requieren más datos.
- **Sesión 7** (Transformers): evolución natural para secuencias.

Contenido

1. Introducción
2. Naive Bayes para Clasificación de Texto
3. Support Vector Machines (SVM) para Texto
4. Modelos Ocultos de Markov (HMM)
5. Modelos de Máxima Entropía (MaxEnt)
6. Conditional Random Fields (CRF)
7. Comparación de Modelos Clásicos
8. Caso Integrado: Clasificación y NER en Reseñas
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Lo que hemos aprendido

Conceptos clave

- **Naive Bayes**: clasificación rápida, asume independencia.
- **SVM**: clasificación robusta, margen máximo, kernel lineal.
- **HMM**: modelado de secuencias generativo, Viterbi, Baum-Welch.
- **MaxEnt**: regresión logística multinomial, características arbitrarias.
- **CRF**: discriminativo para secuencias, supera a HMM en rendimiento.
- **Comparación**: elección según tarea, datos y necesidad de interpretabilidad.
- **Caso práctico**: clasificación y NER con modelos clásicos.

¡Gracias!